



CYBERSECURITY AS A SERVICE

Documentation Technique Complète Plateforme B2B SaaS

Application web e-commerce B2B spécialisée dans la vente de solutions cybersécurité en mode SaaS. Architecture découplée Frontend React / Backend Laravel avec paiement Stripe, génération PDF, emails transactionnels et chatbot IA.

Projet fil rouge BSI Dev — Sup de Vinci 2025-2026

Auteur : Noun — Branche CYNA-Noun

Dépôt : github.com/GeorgeKhita/Projet-CYNA-DEV

Dernier commit : 626c3fe — docs: README complet

Date : Juin 2026

Version : 1.0 — Production-ready MVP

Table des matières

I. Présentation du projet

1. Vue d'ensemble et objectifs 3

2. Stack technique 3

3. Architecture générale 4

II. Installation & Configuration

4. Prérequis 5

5. Installation backend Laravel 5

6. Installation frontend React 6

7. Variables d'environnement 6

III. Base de données

8. Schéma — 17 migrations 7

9. Relations entre modèles 8

IV. Backend Laravel

10. Structure des contrôleurs 9

11. API REST — tous les endpoints 9

12. Authentification Sanctum 11

13. Intégration Stripe 11

14. Emails & Mailtrap 12

15. Génération PDF (dompdf) 12

16. Chatbot IA (Claude Anthropic) 13

V. Frontend React

17. Structure & routing 13

18. Pages utilisateur 14

19. Espace client 15

20. Panel admin 15

21. Composants partagés 16

22. Gestion de l'authentification 16

VI. Flux métier

23. Flux de commande & paiement 17

24. Flux de réinitialisation mot de passe 17

25. Flux chatbot 18

VII. Sécurité & RGPD

26. Middleware & protection des routes 18

VIII. Historique & État du projet

28.Historique des commits19

29.Fonctionnalités complètes vs manquantes20

30.Comptes de démonstration & tests20

I. Présentation du projet

1. Vue d'ensemble et objectifs

CYNA est une plateforme e-commerce B2B dédiée à la vente d'abonnements à des solutions de cybersécurité managées. Le projet vise à proposer une expérience d'achat complète : catalogue de services, panier, paiement en ligne sécurisé, espace client avec gestion des abonnements, et un panel d'administration complet.

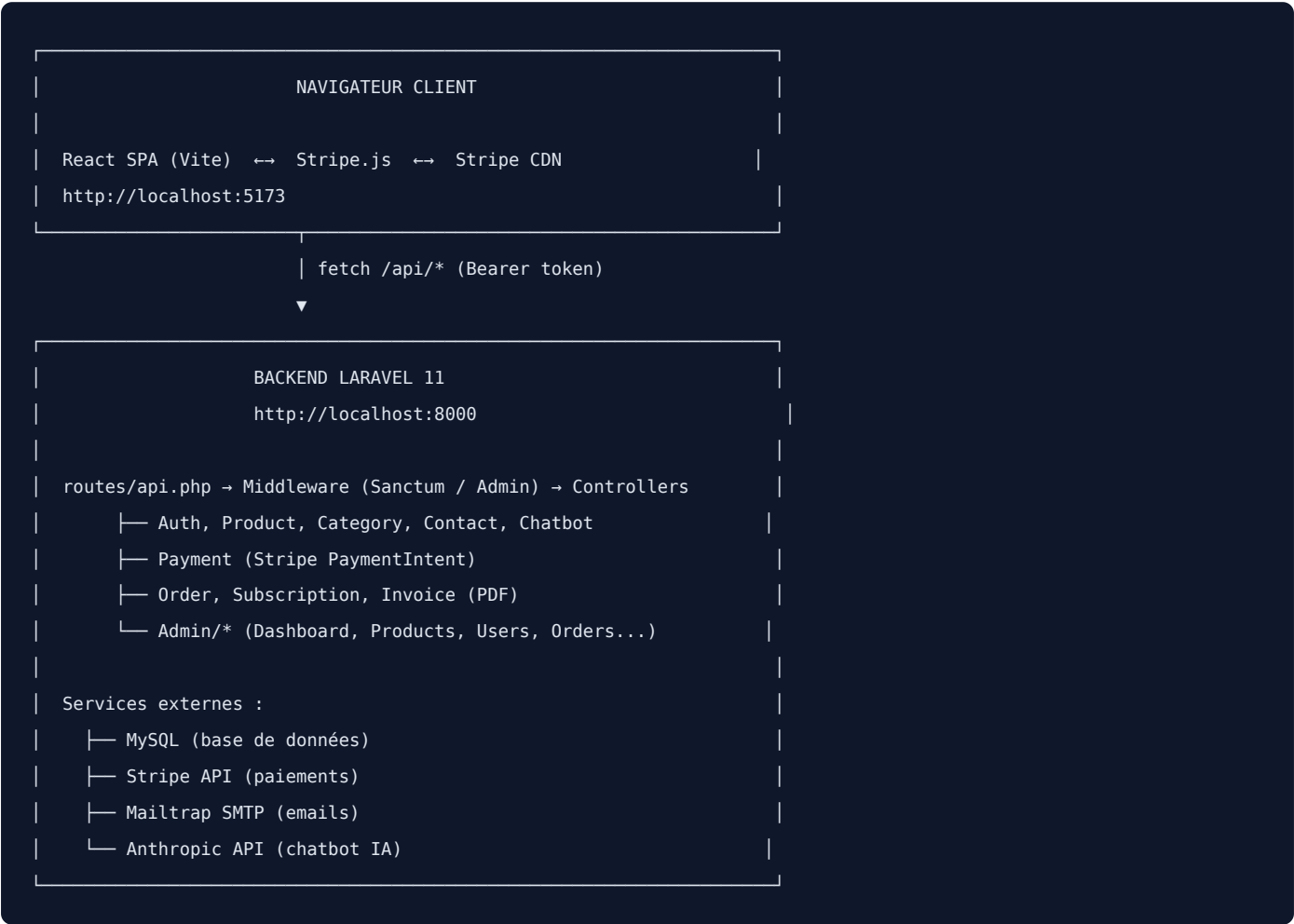
Axe	Détail
Type	Application web fullstack — SPA + API REST
Cible	Entreprises B2B cherchant à externaliser leur cybersécurité
Modèle	Abonnements mensuels ou annuels (SOC, EDR, XDR, SIEM, CTI...)
Architecture	Frontend découplé (React) + Backend API (Laravel)
Déploiement	Frontend : Vite build statique · Backend : PHP/Apache ou Laravel Herd

2. Stack technique

Couche	Technologie	Version	Rôle
Frontend	React	18	Framework UI
Frontend	Vite	6	Bundler & dev server
Frontend	TypeScript	5	Typage statique
Frontend	Tailwind CSS	4	Styles utilitaires
Frontend	React Router	7	Routing SPA
Frontend	shadcn/ui + Radix UI	—	Composants accessibles
Frontend	@stripe/react-stripe-js	—	Widget paiement sécurisé
Frontend	Lucide React	—	Icônes vectorielles
Backend	Laravel	11	Framework PHP
Backend	PHP	8.2+	Runtime serveur
Backend	Laravel Sanctum	—	Authentification API tokens
Backend	MySQL	8+	Base de données relationnelle
Backend	stripe/stripe-php	20.x	SDK paiements Stripe
Backend	barryvdh/laravel-dompdf	3.x	Génération factures PDF
Services	Mailtrap SMTP Sandbox	—	Emails dev/staging
Services	Anthropic Claude API	claude-haiku	Chatbot IA

Couche	Technologie	Version	Rôle
Services	Stripe	—	Paielements en ligne (test + prod)

3. Architecture générale



II. Installation & Configuration

4. Prérequis

Outil	Version minimale	Usage
Node.js	>= 18	Runtime JavaScript frontend
PHP	>= 8.2	Runtime backend
Composer	>= 2.x	Gestionnaire dépendances PHP
MySQL	>= 8	Base de données (WAMP/XAMPP/Herd)
Compte Stripe	—	Clés API test/prod
Compte Mailtrap	—	Sandbox SMTP pour les emails

5. Installation backend Laravel

```
cd backend-laravel
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve
# → Backend disponible sur http://localhost:8000
```

Si les tables existent déjà sans enregistrement dans la table migrations, utiliser le script `fake_migrations.php` (fourni dans le repo) avant de lancer `php artisan migrate`.

6. Installation frontend React

```
cd frontend-react
npm install
# Créer .env avec les variables ci-dessous
npm run dev
# → Frontend disponible sur http://localhost:5173
```

7. Variables d'environnement

Backend — backend-laravel/.env

```
APP_NAME=CYNA
APP_ENV=local
APP_URL=http://localhost:8000
APP_DEBUG=true
```

```
# Base de données MySQL
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cyna
DB_USERNAME=root
DB_PASSWORD=

# CORS / Sanctum
SANCTUM_STATEFUL_DOMAINS=localhost:5173,localhost:5174,127.0.0.1
FRONTEND_URL=http://localhost:5173

# Stripe (clés test)
STRIPE_PUBLIC_KEY=pk_test_51Tfxro...
STRIPE_SECRET_KEY=sk_test_51Tfxro...

# Mailtrap SMTP Sandbox
MAIL_MAILER=smtp
MAIL_HOST=sandbox.smtp.mailtrap.io
MAIL_PORT=2525
MAIL_USERNAME=20aad5c782c158
MAIL_PASSWORD=f0d85c40004c83
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@cyna.local
MAIL_FROM_NAME="CYNA"

# Chatbot IA Claude
ANTHROPIC_API_KEY=sk-ant-api03-...
ANTHROPIC_MODEL=claude-haiku-4-5-20251001
```

Frontend — frontend-react/.env

```
VITE_API_URL=http://localhost:8000
VITE_STRIPE_PUBLIC_KEY=pk_test_51Tfxro...
```

III. Base de données

8. Schéma — 17 migrations

#	Table	Champs principaux	Description
01	users	first_name, last_name, email, password, company, role (user/admin), two_factor_enabled	Comptes utilisateurs
02	categories	name, color	Catégories de produits (SOC, EDR, XDR...)
03	products	category_id, name, slug, description, features (JSON), images (JSON), price_monthly, price_annual, status, priority	Services/produits SaaS
04	addresses	user_id, street, city, postal_code, country	Adresses de facturation
05	payment_methods	user_id, type, last4, brand, stripe_pm_id	Méthodes de paiement
06	carts	user_id	Paniers persistants (optionnel)
07	cart_items	cart_id, product_id, quantity, duration	Lignes de panier
08	orders	user_id, status (pending/paid/failed/refunded), subtotal, tax, total, stripe_pi_id	Commandes
09	order_details	order_id, product_id, quantity, unit_price, total_price, duration (monthly/annual)	Lignes de commande
10	subscriptions	user_id, product_id, order_id, status (active/cancelled), billing_cycle, current_period_start, current_period_end, cancelled_at	Abonnements actifs
11	homepage_carousel	title, subtitle, image_url, cta_text, cta_url, position, active	Slides carousel homepage
12	support_messages	user_id, type (form/chatbot), email, subject, message_body, requires_human, status (new/resolved)	Messages contact & chatbot
13	security_logs	user_id, action, model_type, model_id, details (JSON), ip	Logs d'activité admin
14	site_settings	key, value, type	Paramètres globaux plateforme
15	password_reset_tokens	email, token (hashé), created_at	Tokens reset mot de passe
16	personal_access_tokens	tokenable_id, tokenable_type, name, token, abilities, last_used_at	Tokens Sanctum
17	invoices	user_id, order_id, invoice_number, amount, status (paid/pending/refunded)	Factures PDF

9. Relations entre modèles Eloquent


```
User      → hasMany(Order)
           → hasMany(Subscription)
           → hasMany(Invoice)
           → hasMany(SupportMessage)

Product   → belongsTo(Category)
           → hasMany(OrderDetail)
           → hasMany(Subscription)

Order      → belongsTo(User)
           → hasMany(OrderDetail) [alias: items()]
           → hasOne(Invoice)
           → hasMany(Subscription)

OrderDetail → belongsTo(Order)
            → belongsTo(Product)

Subscription → belongsTo(User)
              → belongsTo(Product)
              → belongsTo(Order)

Invoice    → belongsTo(User)
            → belongsTo(Order)
```

IV. Backend Laravel

10. Structure des contrôleurs

```
app/Http/Controllers/Api/
├─ AuthController.php      register, login, logout, me, updateProfile,
│                           forgotPassword, resetPassword, exportMyData,
│                           sendAdmin2FA, verifyAdmin2FA
├─ ProductController.php   index (filtres), show
├─ CategoryController.php  index
├─ ContactController.php   send
├─ ChatbotController.php   message (IA Claude)
├─ PaymentController.php   createIntent (Stripe PaymentIntent)
├─ OrderController.php     store (vérifie Stripe + crée invoice + email), index, show
├─ SubscriptionController.php index, cancel
├─ InvoiceController.php    index, download (PDF), buildHtml() static
├─ Admin/
│   ├─ DashboardController.php    index (KPIs), revenueChart
│   ├─ AdminProductController.php  CRUD + toggle
│   ├─ AdminCategoryController.php  CRUD
│   ├─ AdminOrderController.php    index, show, updateStatus
│   ├─ AdminUserController.php    index, show, destroy
│   ├─ AdminCarouselController.php  CRUD + reorder
│   ├─ AdminContactController.php  index, show, markResolved, destroy
│   ├─ ActivityLogController.php   index, clear
│   └─ AdminSettingsController.php index, update
```

11. API REST — tous les endpoints

Authentification (public)

Méthode	Endpoint	Description
POST	/api/auth/register	Inscription (first_name, last_name, email, password, company)
POST	/api/auth/login	Connexion → retourne token Sanctum + user
POST	/api/auth/forgot-password	Envoi email avec lien reset (expire 60 min)
POST	/api/auth/reset-password	Réinitialisation avec token + nouveau mot de passe

Authentification (auth:sanctum)

Méthode	Endpoint	Description
POST	/api/auth/logout	Révocation du token courant
GET	/api/auth/me	Profil utilisateur courant
PUT	/api/auth/me	Modifier profil / changer mot de passe

Méthode	Endpoint	Description
GET	/api/auth/me/export	Export RGPD des données (JSON)

Catalogue (public)

Méthode	Endpoint	Description
GET	/api/products	Liste produits (params: category, search, sort, per_page)
GET	/api/products/{id}	Détail d'un produit
GET	/api/categories	Toutes les catégories avec couleurs
POST	/api/contact	Envoyer un message au support
POST	/api/chatbot/message	Envoyer un message au chatbot IA

Commandes & Paiement (auth:sanctum)

Méthode	Endpoint	Description
POST	/api/payments/intent	Créer PaymentIntent Stripe → retourne client_secret
GET	/api/orders	Historique commandes utilisateur (avec invoice_id)
POST	/api/orders	Créer commande après paiement Stripe confirmé
GET	/api/orders/{id}	Détail d'une commande
GET	/api/subscriptions	Abonnements actifs et annulés
PATCH	/api/subscriptions/{id}/cancel	Annuler un abonnement
GET	/api/invoices	Liste des factures
GET	/api/invoices/{id}/download	Télécharger facture en PDF

Panel Admin (auth:sanctum + admin)

Méthode	Endpoint	Description
GET	/api/admin/dashboard	KPIs : revenus, clients actifs, contrats, tickets
GET	/api/admin/dashboard/revenue-chart	Graphique revenus 12 derniers mois
GET	/api/admin/products	Tous les produits (paginés)
POST	/api/admin/products	Créer produit
PUT	/api/admin/products/{id}	Modifier produit
DELETE	/api/admin/products/{id}	Supprimer produit
PATCH	/api/admin/products/{id}/toggle	Activer / désactiver produit
GET	/api/admin/categories	Toutes les catégories

Méthode	Endpoint	Description
POST	/api/admin/categories	Créer catégorie
PUT	/api/admin/categories/{id}	Modifier catégorie
DELETE	/api/admin/categories/{id}	Supprimer catégorie
GET	/api/admin/orders	Toutes les commandes
GET	/api/admin/orders/{id}	Détail commande
PATCH	/api/admin/orders/{id}/status	Changer statut (paid/failed/refunded...)
GET	/api/admin/users	Tous les utilisateurs
GET	/api/admin/users/{id}	Détail utilisateur
DELETE	/api/admin/users/{id}	Supprimer utilisateur
GET	/api/admin/carousel	Slides du carousel
POST	/api/admin/carousel	Créer slide
PUT	/api/admin/carousel/{id}	Modifier slide
DELETE	/api/admin/carousel/{id}	Supprimer slide
POST	/api/admin/carousel/reorder	Réordonner les slides
GET	/api/admin/contact-messages	Tous les messages support
PATCH	/api/admin/contact-messages/{id}/resolve	Marquer résolu
DELETE	/api/admin/contact-messages/{id}	Supprimer message
GET	/api/admin/logs	Logs d'activité
DELETE	/api/admin/logs	Vider les logs
GET	/api/admin/settings	Paramètres plateforme
PUT	/api/admin/settings	Modifier paramètres

12. Authentication Sanctum

L'application utilise Laravel Sanctum pour l'authentification API par tokens. Chaque connexion génère un token personnel stocké en base (table `personal_access_tokens`). Le token est transmis côté client dans le header HTTP `Authorization: Bearer {token}`.

- Tokens révoqués lors de la déconnexion (`currentAccessToken()->delete()`)
- Tous les anciens tokens supprimés à chaque nouvelle connexion
- Redirection automatique vers /connexion si le serveur retourne 401
- Protection CORS configurée pour `localhost:5173` et `localhost:5174`
- Middleware `AdminMiddleware` : vérifie `role === "admin"` sur toutes les routes `/api/admin/*`

13. Intégration Stripe

Le paiement est entièrement géré via Stripe en mode **PaymentIntent** pour une sécurité maximale (PCI-DSS compliant).

FLUX DE PAIEMENT COMPLET :

1. Client arrive sur /checkout/paiement

→ useEffect : POST /api/payments/intent { amount: total }

→ Backend : StripeClient->paymentIntents->create({ amount_cents, currency: "eur" })

→ Retour : { client_secret: "pi_xxx_secret_xxx" }

2. Client saisit sa carte (Stripe CardElement – jamais de données brutes)

→ stripe.confirmCardPayment(client_secret, { payment_method: { card } })

→ Résultat : paymentIntent.status === "succeeded"

3. Client envoie la commande :

POST /api/orders { payment_intent_id: "pi_xxx", items: [...] }

→ Backend : stripe->paymentIntents->retrieve("pi_xxx")

→ Vérification status === "succeeded"

→ Si OK : création Order + OrderDetails + Subscriptions + Invoice

→ Génération PDF + envoi email avec pièce jointe

Carte test	Numéro	Résultat
Visa (succès)	4242 4242 4242 4242	Paiement accepté
Refus générique	4000 0000 0000 0002	Paiement refusé
3DS requis	4000 0025 0000 3155	Authentification 3DS

Date d'expiration : n'importe quelle date future · CVV : 3 chiffres quelconques

14. Emails transactionnels — Mailtrap

Tous les emails de développement/staging sont interceptés par Mailtrap (sandbox SMTP). Aucun email ne part vers de vraies adresses en environnement de test.

Déclencheur	Destinataire	Contenu
Païement confirmé	Client	Email HTML dark-themed + facture PDF en pièce jointe
Mot de passe oublié	Client	Lien sécurisé de réinitialisation (expire 60 min)
Message de contact	support@cyna-it.fr	Email avec le contenu du formulaire + reply-to client

15. Génération PDF — barryvdh/laravel-dompdf

Les factures sont générées à la volée au format PDF via dompdf. La méthode `InvoiceController::buildHtml($invoice)` est une méthode statique partagée entre le téléchargement à la demande et l'attachement email.

- Format A4, police DejaVu Sans (compatible caractères UTF-8)
- Contenu : en-tête CYNA, émetteur/destinataire, tableau des produits, TVA 20%, totaux HT/TTC
- Statut PAYÉE tamponné en vert
- Numérotation : `CYN-000001` (6 chiffres, zero-padded)
- Téléchargeable à tout moment depuis l'espace client (`GET /api/invoices/{id}/download`)

16. Chatbot IA — Claude (Anthropic)

Un assistant virtuel est disponible sur toutes les pages via une bulle flottante. Il répond aux questions sur les produits CYNA en utilisant l'API Claude.

- Base de connaissances couvrant : SOC, EDR, XDR, SIEM, tarifs, période d'essai, conformité (ISO 27001, GDPR, NIS2), intégration, contact
- Messages non reconnus → sauvegardés en base avec `requires_human: true`
- Visibles dans le panel admin (AdminMessagesPage) avec badge "Intervention requise"
- Fonctionne pour les utilisateurs connectés et les visiteurs anonymes

V. Frontend React

17. Structure & routing

```
frontend-react/src/
├─ api/
│   └─ client.ts           Wrapper fetch : Bearer token auto, gestion 401, erreurs
├─ app/
│   └─ components/
│       ├── Layout.tsx     Navbar + Footer + ChatBot + Outlet
│       ├── Navbar.tsx
│       ├── Footer.tsx
│       ├── ChatBot.tsx    Bulle flottante IA
│       ├── DashboardSidebar.tsx  Menu espace client
│       ├── AdminSidebar.tsx  Menu panel admin
│       ├── ProductVisual.tsx Icône colorée par catégorie
│       ├── ErrorBoundary.tsx
│       └─ ui/             ~20 composants shadcn/ui (Button, Card, Dialog...)
│   └─ pages/
│       ├── HomePage.tsx   Accueil
│       ├── CatalogPage.tsx Catalogue + filtres
│       ├── ProductDetailPage.tsx
│       ├── CartPage.tsx
│       ├── CheckoutIdentificationPage.tsx
│       ├── CheckoutPaymentPage.tsx  Stripe CardElement
│       ├── ConfirmationPage.tsx
│       ├── LoginPage.tsx
│       ├── RegisterPage.tsx
│       ├── ForgotPasswordPage.tsx
│       ├── ResetPasswordPage.tsx
│       ├── ContactPage.tsx
│       ├── DashboardPage.tsx
│       ├── AbonnementsPage.tsx
│       ├── CommandesPage.tsx       Avec téléchargement PDF
│       ├── ParametresPage.tsx
│       └─ admin/
│           ├── AdminLayout.tsx
│           ├── AdminDashboardPage.tsx
│           ├── AdminProductsPage.tsx
│           ├── AdminUsersPage.tsx
│           ├── AdminOrdersPage.tsx
│           └─ AdminMessagesPage.tsx
│   └─ routes.tsx
├─ context/
│   └─ AuthContext.tsx      user, token, isAuthenticated, login, logout
└─ lib/
    └─ cart.ts              Panier localStorage + CATEGORY_COLORS
```

18. Pages utilisateur

Route	Page	Fonctionnalités clés
/	HomePage	Hero, stats, catégories dynamiques, top produits depuis API, CTA
/catalogue	CatalogPage	Filtres catégorie, recherche textuelle, tri prix/priorité, ProductVisual
/produit/:id	ProductDetailPage	Description, features, comparatif mensuel/annuel, ajout panier
/panier	CartPage	Items localStorage, ajustement quantités, suppression, total
/checkout/identification	CheckoutIdentificationPage	Connexion / inscription en contexte checkout
/checkout/paiement	CheckoutPaymentPage	Stripe CardElement, création PaymentIntent, confirmation paiement
/confirmation	ConfirmationPage	Récap commande, numéro de commande, prochaines étapes
/connexion	LoginPage	Email + mot de passe, stockage token localStorage
/inscription	RegisterPage	Prénom, nom, entreprise, email, mot de passe confirmé
/mot-de-passe-oublie	ForgotPasswordPage	Envoi email avec lien reset
/reinitialiser-mot-de-passe	ResetPasswordPage	Token + email en query params, nouveau mot de passe
/contact	ContactPage	Formulaire email/sujet/message, envoi support

19. Espace client (/espace-client/*)

Route	Page	Fonctionnalités
/espace-client	DashboardPage	KPIs : abonnements actifs, coût mensuel total, prochain renouvellement
/espace-client/abonnements	AbonnementsPage	Liste actifs/annulés, badge couleur par catégorie, bouton annulation
/espace-client/commandes	CommandesPage	Historique avec statuts, détail expandable, bouton téléchargement PDF
/espace-client/parametres	ParametresPage	Modifier prénom/nom/entreprise/email, changer mot de passe, export RGPD

20. Panel admin (/admin/*)

Route	Page	Fonctionnalités
/admin	AdminDashboardPage	KPIs revenus, clients actifs, contrats, tickets · Tableau commandes récentes · Graphique revenus
/admin/produits	AdminProductsPage	CRUD complet · Édition features (textarea JSON) · Toggle actif/inactif · Prix mensuel/annuel
/admin/utilisateurs	AdminUsersPage	Liste avec recherche · Détail par user (commandes + abonnements) · Suppression
/admin/commandes	AdminOrdersPage	Toutes commandes · Détail expandable · Changement statut via dropdown

Route	Page	Fonctionnalités
/admin/messages	AdminMessagesPage	Messages chatbot + contact · Badge "Intervention requise" · Marquer résolu · Supprimer

21. Composants partagés

- **Layout.tsx** — wraps toutes les pages publiques/client : Navbar + ChatBot flottant + Footer + Outlet
- **ChatBot.tsx** — bulle flottante en bas à droite, historique messages, questions suggérées, appel API Claude
- **ProductVisual.tsx** — icône colorée correspondant à la catégorie du produit (Shield=SOC, Laptop=EDR, Globe=XDR...)
- **DashboardSidebar.tsx** — navigation espace client + déconnexion
- **AdminSidebar.tsx** — navigation panel admin
- **ErrorBoundary.tsx** — capture les erreurs React et affiche une page de repli

22. Gestion de l'authentification

L'authentification est gérée via un Context React (`AuthContext.tsx`) qui expose :

- `user` — objet utilisateur courant (id, name, email, role)
- `isAuthenticated` — booléen
- `loading` — état de chargement initial
- `login(token, user)` — stocke en localStorage
- `logout()` — vide localStorage + appelle `POST /api/auth/logout`

Le client API (`api/client.ts`) intercept automatiquement les réponses 401 et redirige vers `/connexion` .

VI. Flux métier

23. Flux de commande & paiement complet

```
| 1. /checkout/paiement (CheckoutPaymentPage) |
|   ↳ useEffect au montage |
| 2. POST /api/payments/intent { amount: total } |
|   ↳ Backend : Stripe::paymentIntents->create(...) |
| 3. ← { client_secret: "pi_xxx_secret_xxx" } |
| | |
| 4. Client remplit CardElement (données ne quittent pas Stripe) |
|   ↳ clic "Confirmer l'achat" |
| 5. stripe.confirmCardPayment(client_secret, { card }) |
|   ↳ Stripe traite le paiement |
| 6. ← paymentIntent.status === "succeeded" |
| | |
| 7. POST /api/orders { |
|   payment_intent_id: "pi_xxx", |
|   subtotal, tax, total, |
|   items: [{ product_id, qty, unit_price, duration }] |
| } |
|   ↳ Backend vérifie : stripe->paymentIntents->retrieve() |
|   ↳ status === "succeeded" ? continuer : 422 |
| | |
| 8. DB::beginTransaction() |
|   ├── Order::create(...) → order CYN-000001 |
|   ├── OrderDetail::create(...) × n items |
|   ├── Subscription::create(...) × n items |
|   ├── Invoice::create(...) → CYN-000001 |
|   └── DB::commit() |
| | |
| 9. Pdf::loadHtml(InvoiceController::buildHtml($invoice)) |
|   ↳ Mail avec PDF attaché → Mailtrap sandbox |
| | |
| 10. ← { id, ref, status:"paid", total, invoice_id } |
|     ↳ Frontend : clearCart() + navigate("/confirmation")
```

24. Flux de réinitialisation de mot de passe

```
1. POST /api/auth/forgot-password { email }
   ↳ User existe ? continuer (sinon même réponse pour éviter l'énumération)
2. Génération token aléatoire (64 chars) + hash bcrypt stocké en DB
3. Email envoyé : lien http://localhost:5173/reinitialiser-mot-de-passe?token=xxxx&email=user@example.com
4. Lien valide 60 minutes
```

5. POST /api/auth/reset-password { token, email, password, password_confirmation }
 - ↓ Récupère enregistrement par email
 - ↓ Vérifie : délai 60 min + Hash::check(token, stored_hash)
 - ↓ Nouveau mot de passe hashé + sauvegardé
 - ↓ Token supprimé + tous les tokens Sanctum révoqués

25. Flux chatbot IA

1. Utilisateur envoie un message (connecté ou anonyme)
POST /api/chatbot/message { message, history: [...] }
2. ChatbotController détecte le topic dans le message :
SOC | EDR | XDR | SIEM | prix | essai | conformité |
intégration | contact | connexion | commande | bonjour
3. Si topic reconnu → réponse depuis base de connaissances statique
Si non reconnu → appel API Claude (Anthropic) avec contexte CYNA
4. SupportMessage::create({ type:"chatbot", requires_human: true/false })
5. ← { message, requires_human }
6. Si requires_human === true → visible dans admin/messages avec badge orange

VII. Sécurité & RGPD

26. Middleware & protection des routes

Niveau	Protection	Routes concernées
Public	Aucune	/api/auth/register, /api/auth/login, /api/products, /api/categories, /api/contact, /api/chatbot/message
Authentifié	auth:sanctum	/api/orders, /api/subscriptions, /api/invoices, /api/payments/intent, /api/auth/me, /api/auth/logout
Admin	auth:sanctum + admin	Toutes les routes /api/admin/*

AdminMiddleware — vérifie que `$request->user()->role === "admin"`, retourne 403 sinon.

Validation stricte — toutes les entrées sont validées avec les règles Laravel (type, min/max, exists, in, starts_with...).

Paie ment — les données de carte ne transitent jamais par le backend. Stripe héberge le widget CardElement en iframe sécurisée. Le backend ne reçoit qu'un `payment_intent_id` et le vérifie directement auprès de l'API Stripe.

Tokens — révoqués à chaque déconnexion. En cas de nouvelle connexion, tous les anciens tokens sont supprimés.

27. Conformité RGPD

Droit	Endpoint	Statut
Droit d'accès	GET /api/auth/me	Implémenté
Portabilité des données	GET /api/auth/me/export (JSON)	Implémenté
Rectification	PUT /api/auth/me	Implémenté
Droit à l'oubli	DELETE /api/auth/me	À implémenter

L'export RGPD (GET /api/auth/me/export) retourne un fichier JSON nommé `mes-donnees-cyna-YYYY-MM-DD.json` contenant : profil, commandes et abonnements.

VIII. Historique & État du projet

28. Historique des commits (branche CYNA-Nouh)

626c3fe	docs: README complet — stack, installation, API, flux paiement, historique
fe25521	feat: intégration Stripe, Mailtrap, factures PDF et corrections
3bdcef0	fix: corriger méthodes HTTP dans le panel admin
0e69bb2	feat: images produits + panel admin complet
8cb4f23	feat: priorité haute — toutes les pages manquantes implémentées
29cf3a8	fix: redirection automatique vers /connexion si token expiré (401)
78fa8ef	feat: pages espace-client complètes + corrections backend
403edd9	fix: corriger SubscriptionController pour le nouveau schéma
7c0f684	fix: corriger Order/OrderDetail/Subscription et OrderController pour le nouveau schéma
709fc7c	docs: ajouter documentation de collaboration front/back
6f79dc0	config: URL backend configurable via variable d'environnement
273e2b4	fix: corriger Product/Category/ProductController pour le nouveau schéma
9e6bbde	fix: renommer routes.ts en routes.tsx (JSX non supporté en .ts)
cad9624	fix: corriger les routes manquantes et stabiliser la navigation
8224f36	fix: corriger l'erreur removeChild et stabiliser l'application
7c3781c	fix: nettoyer AuthController et messages d'erreur de validation
b1f525c	fix: mettre à jour le modèle User pour correspondre au nouveau schéma BDD
0c90ce8	feat: connexion complète front-end / back-end
b78b540	feat: implémentation du chatbot IA avec l'API Claude (Anthropic)
49b9714	refactor: remplacer les migrations Laravel par le schéma cyna_corrected.sql
db2a851	refactor: réorganiser le frontend dans le dossier frontend-react/
c3143b5	docs: add complete README with stack, install guide, routes and API
9a7f7ad	feat: compléter fonctionnalités manquantes, pages légales, 2FA, RGPD
61810b8	feat: chatbot global CYNA avec base de connaissance SOC/EDR/XDR
56f7288	feat: implémentation complète de toutes les fonctionnalités specs
1b9136b	feat: backend Laravel complet + couche API front-end
bf4e924	feat: MVP complet React + panel admin CYNA
a721b5a	feat: import maquette Figma et améliorations UI/UX
ffcd7ae	first commit

29. Fonctionnalités — état complet

Fonctionnalité	Statut	Notes
Catalogue produits + filtres	Complet	API dynamique, recherche, tri, categories
Détail produit + panier	Complet	localStorage, quantités, totaux
Inscription / Connexion	Complet	Sanctum tokens, validation complète
Reset mot de passe	Complet	Email + token 60 min
Paieement Stripe	Complet	PaymentIntent, CardElement, vérification serveur
Confirmation commande	Complet	Email + PDF joint automatiquement
Factures PDF	Complet	dompdf, téléchargeable depuis espace client
Abonnements	Complet	Création auto, annulation, historique
Espace client complet	Complet	Dashboard, abonnements, commandes, paramètres
Export RGPD	Complet	JSON avec profil + commandes + abonnements
Panel admin complet	Complet	Dashboard KPIs, CRUD produits, users, orders, messages
Chatbot IA	Complet	Claude API + KB statique + escalade admin
Contact support	Complet	Formulaire + email + stockage DB
Carousel homepage	Partiel	API admin OK, frontend non affiché
Webhooks Stripe	Manquant	Synchronisation asynchrone à implémenter
Renouvellement abonnements	Manquant	Cron Laravel à implémenter
Droit à l'oubli RGPD	Manquant	DELETE /api/auth/me à implémenter
2FA Admin	Stub	Endpoints existent, retournent 501

30. Comptes de démonstration & tests

Rôle	Email	Mot de passe
Admin	admin@cyna-it.fr	Admin1234!
Utilisateur	jean.dupont@entreprise.fr	User1234!

État global du projet : MVP Production-Ready (~85% complet)
Toutes les fonctionnalités cœur de métier sont opérationnelles. Les éléments manquants (carousel frontend, webhooks Stripe, cron abonnements, droit à l'oubli) n'impactent pas le parcours d'achat principal.